SOFTWARE REQUIREMENTS SPECIFICATION (SRS) v0.0.1

Project: AMC Movie Coordination Service (Discord Integration)

Document Version: v0.0.1

Status: Approved for Development

1. INTRODUCTION

1.1 Purpose

This document defines the architectural and functional specifications for a custom Discord bot integration. The service is designed to facilitate group coordination for cinematic events at AMC locations by aggregating local showtimes, synthesizing third-party review data, and executing automated consensus-gathering (polling) mechanisms within a server environment.

1.2 Scope

The application operates as an asynchronous, event-driven microservice interfacing directly with the Discord API. To circumvent the limitations of AMC's B2B API access, the service employs a Data Ingestion and DOM Parsing module to extract theater heuristics and runtime schedules dynamically from public endpoints. The system relies on local state caching for performance optimization, features role-based access control (RBAC) for configuration, and utilizes parameter-driven payload visibility to manage user interface (UI) clutter.

1.3 Definitions and Acronyms

- **DOM:** Document Object Model
- **RBAC:** Role-Based Access Control
- **TTL:** Time-to-Live (referencing cache invalidation)
- **OMDb:** Open Movie Database
- **CLI:** Command Line Interface (mapped to Discord Slash Commands)

2. SYSTEM ARCHITECTURE

2.1 Technology Stack

- **Runtime Environment:** Python 3.10+
- **Asynchronous Framework:** asyncio / aiohttp (Non-blocking I/O operations)
- **API Wrapper:** discord.py (WebSocket and REST API communication)
- **DOM Traversal:** BeautifulSoup4 (HTML parsing)
- **State Management:** Ephemeral JSON-based document storage (Configuration and Cache)

2.2 High-Level Architecture

The system is divided into three decoupled components:

1. **Ingestion Service (Web Scraper):** An independent module responsible for network requests, HTML parsing, and exception handling against structural DOM mutations.
2. **Discord Gateway (Event Loop):** The primary WebSocket connection handling command routing, autocomplete generation, and payload construction.
3. **Task Scheduler:** A daemonized background process handling TTL-based cache invalidation and scheduled poll execution.

3. FUNCTIONAL REQUIREMENTS (FRs)

3.1 Epic: Data Ingestion & Synchronization

- **FR-1.1:** The service shall execute a scheduled GET request to the target XML sitemap or directory index (TBD) to extract geographical and URL string identifiers for all active theater locations.
- **FR-1.2:** The service shall serialize and commit the extracted theater heuristics to a local JSON cache to minimize external network calls and prevent rate-limiting.
- **FR-1.3:** The service shall parse target DOM structures on demand to extract daily operational schedules, mapping movie titles to aggregate showtime arrays.
- **FR-1.4:** The service shall implement fault tolerance for DOM mutation exceptions. Upon AttributeError or parsing failure, the system shall construct and dispatch an alert payload via Direct Message (DM) to the designated Server Administrator.

3.2 Epic: Configuration & State Management (Admin)

- **FR-2.1:** The service shall expose a /set_theater command restricted to users possessing server administration permissions (RBAC validation).
- **FR-2.2:** The service shall leverage the Discord API autocomplete callback, querying the local JSON cache to provide sub-second typeahead suggestions during theater selection.
- **FR-2.3:** The service shall persist the selected location's unique URL identifier to a local configuration state file.
- **FR-2.4:** A scheduled asynchronous task shall execute weekly to invalidate and refresh the theater cache, ensuring delta synchronization for newly operational or decommissioned locations.

3.3 Epic: Client Interfaces & Query Resolution

Note: All client-invoked commands must default to an ephemeral state (private UI payload) to optimize chat cleanliness, utilizing an optional overriding string parameter for state transition.

- **FR-3.1:** The service shall expose a /movies command that triggers the ingestion module to fetch, parse, and embed the current day's operational schedule for the configured location.
- **FR-3.2:** The service shall expose a /showtimes command accepting two primary arguments: movie (leveraging typeahead autocomplete bounded to the current daily schedule) and date (validating an mm/dd string format and inferring the current calendar year computationally).
- **FR-3.3:** The service shall expose a /upcoming command to parse and return future theatrical releases mapped to the active configuration state.
- **FR-3.4:** The service shall expose a /review command that queries the external OMDb REST API, parses the JSON response, and constructs a Discord Embed containing synthetic aggregate scores (Rotten Tomatoes, Metacritic).
- **FR-3.5:** All client commands shall accept an optional string parameter visibility.
- **FR-3.6:** The gateway shall evaluate the visibility parameter. If the string evaluates strictly to visable, the gateway shall set ephemeral=False, broadcasting the payload to the channel. In all other cases (including null/None), the gateway shall default to ephemeral=True.

3.4 Epic: Automated Event Scheduling

- **FR-4.1:** The service shall initialize a background daemon utilizing discord.ext.tasks.
- **FR-4.2:** On a configurable, recurring cron schedule (weekly), the daemon shall invoke the ingestion module targeting a forward-looking date parameter (upcoming week).
- **FR-4.3:** The service shall parse the aggregate showtime data, select a subset of highly-indexed operational titles, and construct a native Discord UI Poll object.
- **FR-4.4:** The service shall dispatch the Poll object to a pre-configured, persistent channel ID designated for group consensus.

4. NON-FUNCTIONAL REQUIREMENTS (NFRs)

4.1 Performance & Concurrency

- **NFR-1 (Asynchronous I/O):** All external HTTP requests and file I/O operations must be awaited to prevent blocking the primary event loop, ensuring concurrent processing of multitenant commands.
- **NFR-2 (Latency):** Autocomplete queries querying the local JSON cache must resolve and return payloads to the Discord API within 500ms to ensure a seamless UI/UX.

4.2 Fault Tolerance & Graceful Degradation

- **NFR-3 (Error Handling):** In the event of an external API timeout, 4xx/5xx status code, or DOM parsing failure, the bot shall catch the exception and return a standardized, user-facing error embed rather than terminating the runtime process.

- **NFR-4 (Dependency Resilience):** The discord gateway and internal event loops must remain fully operational even if the AMC web properties are temporarily unreachable.

4.3 Security & Secrets Management

- **NFR-5 (Environment Variables):** Hardcoding of API keys, Discord tokens, or infrastructure secrets is strictly prohibited. All environment variables must be injected at runtime via an external .env file or CI/CD secrets manager.
- **NFR-6 (Least Privilege):** The application shall request only the necessary Discord OAuth2 scopes (e.g., application.commands, send_messages) and abstain from requesting elevated intents (like message_content) unless strictly required by the architecture.